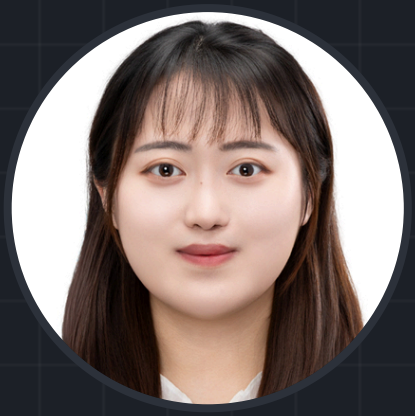
CRM · JAVA BACKEND DEVELOPER 지원

하지혜



Every Record Counts — 고객의 기록이 정확하게 쌓이고, 끊김 없이 이어지도록.

백엔드 경력 약 3년. B2B SaaS의 고객 계정·권한·구독 결제·어드민을 **직접 설계·운영**하며
고객 CS를 **구조 개선**으로 연결해 왔습니다.

hajh5447@naver.com · 010-2882-5447 · dev-annieha.tistory.com · github.com/AnnieHa1002

커리어 타임라인 2022 — 2026

2022.08
한국외국어대학교 졸업
언어인지과학과 · 부전공 통계학

2022.11 – 2025.09 · 2년 11개월
콘파파 Backend Engineer
B2B SaaS REAT 백엔드 단독 설계·운영
— 고객·권한·결제·어드민 전 영역

2026.02 – 2026.05
KDT AI 백엔드 아키텍처 심화
팀장 · 고글에듀 최종 프로젝트 우수상

2026.09 – 진행 중
나공주 키우기 백엔드 설계
여러 기기 데이터 병합 · 중복 지급 방지
(2027.03 출시 예정)

TECH STACK

Java · Spring Boot | PostgreSQL · Redis | JavaScript · React | AWS · GCP · Docker

운영자 화면까지 직접 만듭니다.

WHY CRM

고객 관계의 뼈대가 되는 데이터를 설계하고 운영해 온 개발자

지원동기

고객과 회사 사이의 모든 약속은 결국 데이터로 남습니다.

콘파파에서 B2B SaaS REAT의 백엔드를 혼자 맡아, 고객사 계정·권한·구독 결제·라이선스처럼 고객 관계의 뼈대가 되는 데이터를 직접 설계했습니다. 특수 계약 고객의 라이선스가 캐시 때문에 반영되지 않았던 일을 겪으며, **고객 데이터 한 줄의 오류가 곧 신뢰의 문제**라는 것을 배웠습니다.

운영팀이 쓰는 어드민 화면과 지표 대시보드도 직접 만들었습니다. 고객을 관리하는 사람의 불편을 줄이는 일이 결국 고객 경험으로 이어진다는 점에서, CRM은 제가 해 온 일을 가장 넓게 펼칠 수 있는 분야입니다.

언어인지과학을 전공하며 언어를 사람을 잇는 도구로 바라보게 됐습니다. **고객과 기업을 잇는 시스템을 Java와 웹 화면으로 단단하게 만들고 싶습니다.**



WHAT CRM NEEDS	WHAT I BRING
고객·계정 데이터 모델링	✓ 워크스페이스 멀티테넌시 · 다층 권한, 권한 판단 중앙화 (REAT)
계약·구독·결제 관리	✓ PortOne 구독 결제 5종 이상, 쿠폰·레퍼럴 · 다통화(KRW·USD·JPY)
운영자용 화면(백오피스)	✓ React 어드민 — 라이선스 관리, 워크스페이스 이전, 엔터프라이즈 맞춤
통계·리포트	✓ 사용자 활동 데이터 자동 수집, Looker Studio 지표 대시보드
고객 커뮤니케이션	✓ SSE 실시간 알림 · 끊긴 알림 재전송, FCM 즉시·배치·예약 발송
데이터 정합성	✓ 결제 보상 트랜잭션, 계약 정보 캐시 무효화, Kafka Outbox/Inbox (p.4)
(우대) 대량 데이터 조회 성능	✓ 리스트 쿼리 75 → 5개, 검색 응답 726 → 163ms (p.3)

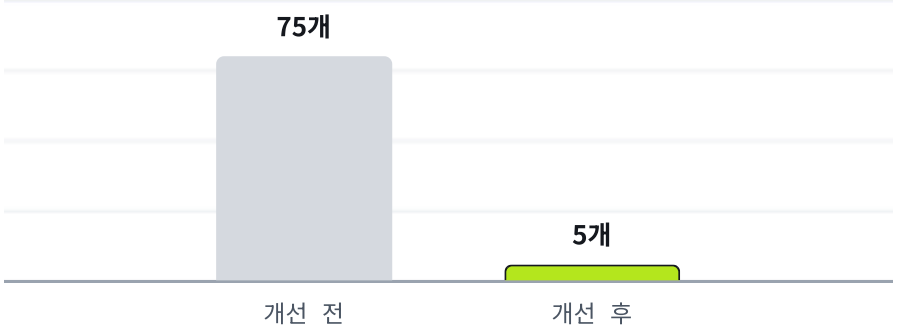
IMPACT

감이 아니라 숫자로 증명한 개선

Before After

N+1 해소 — 리스트 API 쿼리 수

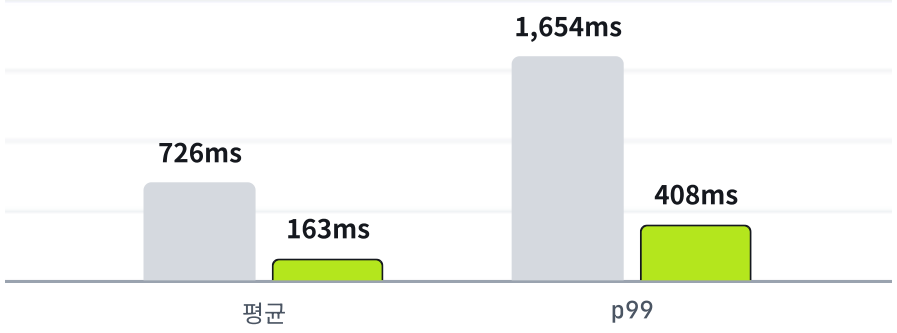
-93%



JOIN FETCH로 N+1을 해소하고 Cache-Aside Redis 캐시를 더했습니다. Redis 장애 시 DB로 자동 전환되는 Fail-safe 구조. (REAT)

검색 성능 — 응답시간

-77%



B-Tree·Elasticsearch·FTS를 비교해 pg_trgm GIN 인덱스를 채택했습니다. 117,000건, VU 50 부하 테스트 기준. (고글에듀)

대용량 이미지 처리 — 20장 처리 시간

-95%



이미지 처리를 Lambda로 분리하고 CompletableFuture로 병렬화했습니다. 2GB 이상 업로드 시의 OOM도 해소. (REAT)

인프라 비용 — 월 고정비



~~월 150만원~~

→ 월 3만원

AWS에서 GCP로 단계 이전했습니다. 데이터 손실 0건, 전환 작업 각 4시간 이내. (REAT)

CUSTOMER DOMAIN

고객 한 명의 생애주기를 따라 설계한 것들

01 · 가입
워크스페이스 단위 멀티테넌시

→

02 · 권한
다층 권한, 판단 중앙화

→

03 · 결제
빌링키 구독, 웹훅 반영

→

04 · 계약
라이선스, 쿠폰, 레퍼럴

→

05 · 소통
SSE 실시간 알림, 수신 설정

→

06 · 운영
React 어드민, Looker 지표

실제 사례 — 고객 데이터가 어긋나지 않게 지킨 세 번의 결정

권한·결제·계약은 한 번 틀어지면 고객 신뢰가 바로 흔들리는 데이터입니다. REAT를 운영하며 직접 겪고 고친 사례입니다.

02 · 권한

흩어진 판단을 한 곳으로

문제

BM 변경으로 권한을 고치자, 흩어진 판단 로직에서 오류가 이어졌습니다.

결정

다층 권한을 재설계하고 모든 판단을 한 곳에서만 처리하게 했습니다.

결과

권한 오류가 거의 사라졌고, 새 기능은 한 곳만 고치면 됩니다.

03 · 결제

실패해도 어긋나지 않게

문제

갱신·플랜 변경·쿠폰·재청구가 얽혀 PG와 DB가 어긋날 수 있었습니다.

결정

결제 정보를 먼저 저장해 웹훅으로 반영하고, 실패하면 보상 트랜잭션.

결과

결제 오류 3건(네트워크 제외 1건), 모두 정합성 유지.

04 · 계약

특수 계약 고객도 빠짐없이

문제

캐시 도입 후 특수 계약 고객의 라이선스가 실서비스에서만 누락됐습니다.

결정

계약 데이터가 바뀌는 모든 지점에서 캐시를 즉시 무효화했습니다.

결과

같은 유형의 오류는 다시 발생하지 않았습니다.

"고객 데이터 한 줄의 오류는 곧 신뢰의 문제다. 정합성을 먼저 설계한다."

"어드민 화면까지가 기능이다. 운영자가 매번 개발자를 찾지 않게 만든다."

"같은 CS가 두 번 오지 않게, 증상이 아니라 구조를 고친다."

ARCHITECTURE & DECISIONS

여러 곳에서 들어온 데이터를 하나의 기록으로

결제 파이프라인

5종 이상의 결제 시나리오를 하나의 흐름으로 통합 (REAT)

PortOne 웹훅 수신 → 결제 실행 / 구독 상태 분리 → **실패 시 보상 트랜잭션**

다중 기기 데이터 병합 (설계 단계)

나공주 키우기 — 기기에서 먼저 보인 보상을 서버가 확정·병합

기기 A / 기기 B → 같은 멱등키로 서버 재정산 → **DB 유일 제약 1회 지급 · 원장 기록**

기술 의사결정 — 왜 이렇게 했는지 대안을 비교해 고른 기록

주제	대안 → 선택	선택 이유
실시간 알림	Polling / WebSocket / SSE → **SSE**	단방향 알림이면 충분하고 운영 부담이 작음. 끊긴 알림은 Redis로 재전송
검색	B-Tree / Elasticsearch / FTS / GIN+Trigram → **GIN+Trigram**	117,000건에서 별도 검색 인프라 없이 p99 408ms
동시성	비관적 락 / 낙관적 락 / Redisson Watchdog → **Watchdog**	커넥션 점유·충돌 실패 없이 인원 초과·중복 신청 차단
Redis 고가용성	단일 / Cluster / Managed / Sentinel → **Sentinel**	현재 규모에 맞는 운영 복잡도로 자동 Failover 확보
수신 설정	Boolean 컬럼 7개 / 비트마스크 → **비트마스크**	알림 설정 컬럼 7개를 1개로 줄여 스키마 단순화

고객 데이터의 **정합성 · 추적 가능성 · 운영 편의성**을 기준으로 설계합니다.
지금은 나공주 키우기에서 여러 기기 데이터 병합과 원장 기반 정산을 설계하고 있습니다.

APPENDIX A — 본편에 담지 못한 문제 해결 기록

고객 운영 도구와 알림

어드민·운영 데이터 시스템 콘파파 · REAT·TOONDA · React 풀스택

문제

운영팀이 고객사 라이선스·워크스페이스를 관리하고 서비스 지표를 볼 내부 도구가 없었습니다.

대응

React 어드민 웹과 API를 개발해 라이선스 관리, 워크스페이스 이전, 엔터프라이즈 맞춤 기능을 넣었습니다. 사용자 활동 데이터는 자동 수집해 Looker Studio 대시보드로 시각화했습니다.

결과

사용량·운영 지표로 의사결정을 지원했고, 사용률 20% 수준이던 어드민·개발 서버 운영비를 80% 이상 줄였습니다.

실시간 알림과 수신 설정 콘파파 · REAT · SSE · Redis

문제

1분 Polling은 주기를 줄이면 요청이 급증하고, 늘리면 알림이 늦었습니다. 업로드 진행률 CS도 늘었습니다.

대응

단방향 알림에 맞는 SSE를 WebSocket 대신 채택하고, Redis에 사용자별 마지막 알림 ID를 저장해 재연결 시 재전송했습니다. 같은 구조를 진행률에도 쓰고, 수신 설정 7종은 비트마스크 1개로 통합했습니다.

결과

반복 요청을 없애고, 연결이 끊겨도 알림 유실이 없었습니다. 수신 설정 컬럼은 7개에서 1개로 줄었습니다.

FCM 알림 파이프라인 Trobl · 사이드 프로젝트

문제

즉시 전달, 단시간 다수 발생, 정기 발송 알림을 하나의 구조로 처리해야 했습니다.

대응

즉시·배치·예약 전략을 분리했습니다. 몰리는 알림은 Redis List에 쌓아 10분마다 사용자별 1건으로 묶고, 예약은 시간대별로 저장해 1분 스케줄러가 보냈습니다. 토큰 오류는 DB에서 자동 삭제했습니다.

결과

FCM 트래픽과 발송 실패율이 줄었고, 별도 메시지 큐 없이 기존 Redis로 세 전략을 통합했습니다.

APPENDIX B — 본편에 담지 못한 문제 해결 기록

정합성과 동시성

Redis 동시성 제어와 장애 대응 Redisson Watchdog · Fail-safe · Sentinel

문제

멀티 인스턴스에서 멘토링 인원 차감과 신청 생성을 함께 지켜야 했고, 캐시·락을 맡은 Redis가 단일 장애점이었습니다.

대응

SETNX 직접 구현 대신 Redisson Watchdog으로 lease를 자동 연장했습니다. REAT는 Redis 장애 시 DB로 넘어가는 Fail-safe를, 고글에듀는 Redis 3대·Sentinel 3대의 자동 Failover를 구성했습니다.

결과

모집 인원 초과와 중복 신청을 막았고, Redis 장애 중에도 서비스가 정상 동작했습니다.

Kafka Outbox/Inbox 이벤트 처리 팔로우미 배송 · 팀 프로젝트

문제

DB 커밋과 이벤트 발행이 분리돼, Kafka 장애 시 이벤트가 유실되거나 중복될 수 있었습니다.

대응

같은 트랜잭션에서 Outbox에 저장한 뒤 발행하고, 실패하면 스케줄러가 재발행해 3회 실패 시 보상 트랜잭션을 실행했습니다. 소비자는 이벤트 ID를 Inbox 기본키로 저장해 멱등 처리했습니다.

결과

at-least-once 전달과 소비자 멱등성을 확보했고, 공통 라이브러리로 전 서비스에 적용했습니다.

다중 기기 정산·중복 지급 방지 나공주 키우기 · 설계 단계 (2인)

문제

기기가 먼저 보상을 보여 주고 서버가 나중에 확정해, 재전송이나 두 기기 동시 달성 시 중복 지급 위험이 있었습니다.

대응

기기와 서버가 같은 멱등키(session:{id}, plan:{id})를 만들고, DB 유일 제약으로 한 번만 지급하도록 설계했습니다. 결과가 갈리는 위험 6곳(난수 배수, 벽시계 의존 등)도 진단했습니다.

결과

서버 확정값이 다를 때 원장에 조정 줄을 남기고 알리는 정책을 제안했습니다. 서버 구현은 다음 단계입니다.

APPENDIX C — 본편에 담지 못한 문제 해결 기록

아키텍처 전환과 기술 역량

REAT 아키텍처 전환 콘파파 · Lambda 74개 → Spring Boot 중앙 서버

문제

Lambda 함수 약 74개로 나뉜 구조에서 정합성 문제와 콜드 스타트 지연 CS가 이어졌습니다.

대응

Spring Boot 중앙 서버와 JWT 인증을 제안했고, 일정 우려에는 3일 PoC로 서버·로그인을 구현해 전환을 이끌었습니다.

결과

콜드 스타트가 사라지고 정합성 문제를 구조적으로 해결했습니다.

GCP 마이그레이션 · CI/CD 콘파파 · AWS → GCP · Jenkins

문제

월 150만원의 AWS 고정비, 수작업 배포로 잘못된 이미지·환경 변수 누락이 반복됐습니다.

대응

DB → 백엔드 → 프론트 순으로 이전(pg_dump/pg_restore, DNS 전환)하고, GitHub Actions와 Jenkins로 배포를 표준화했습니다.

결과

월 150만원 → 3만원(98%), 데이터 손실 0건, 전환 각 4시간 이내. 배포 실수 제거.

교육 · 학력 · 어학

KDT AI 활용 백엔드 아키텍처 심화 과정
2026.02–2026.05 · 팀장 · 고글에듀 최종 프로젝트 우수상

스파르타 항해99 8기 Java Spring 과정 수료
2022.07–2022.10 · 감사상·소통왕상

한국외국어대학교 언어인지과학과 졸업
2016.03–2022.08 · 부전공 통계학

OPIc 영어 IH
2025.12

기술 스택 총람

언어	Java, JavaScript
프레임워크	Spring Boot, Spring Security, JPA, QueryDSL, jOOQ
프론트엔드	CSS, React(어드민 웹), SSE(EventSource)
데이터	PostgreSQL, MySQL, Redis, Kafka
클라우드	AWS (EC2·RDS·S3·Lambda), GCP (Compute Engine·Cloud SQL)
인프라·CI	Docker, Jenkins, GitHub Actions
관측·테스트	Prometheus, Grafana, Loki, Zipkin, K6, Gatling, JUnit5